

Milestone 1: Sequential HMM-Based Speech Recognition Pipeline

Technical Documentation & Results

Parallel & Distributed Computing Course

Fahad Samad | Fahad Shah Khan | Muhammad Hussain | Saman Waseem

1. Objective

The goal of Milestone 1 is to implement a complete, correct, and well-benchmarked sequential speech recognition pipeline using Hidden Markov Models (HMMs). This pipeline serves as the reference baseline against which the parallel and distributed implementations in Milestones 2 and 3 will be compared for correctness and speedup.

2. System Overview

2.1 Pipeline Flow

The end-to-end pipeline transforms raw audio into a predicted phoneme label through the following stages:

Audio File (.WAV) → MFCC Feature Extraction → Normalisation → HMM Scoring → Viterbi Decoding → Predicted Phoneme

Each stage is implemented as a separate Python module, making the codebase modular and easy to extend for parallelisation in later milestones.

2.2 File Structure

The implementation is organised into five Python modules:

File	Purpose	Key Contents
features.py	Audio feature extraction	<code>extract_mfcc()</code> , <code>normalize_features()</code> , <code>generate_synthetic_features()</code>
hmm.py	Core HMM model	GaussianHMM class: <code>_forward()</code> , <code>_backward()</code> , <code>fit()</code> , <code>score()</code>
viterbi.py	Sequence decoding	<code>viterbi_decode()</code> , <code>classify_utterance()</code> , <code>compress_state_sequence()</code>
pipeline.py	End-to-end orchestration	SpeechRecognizer: <code>train()</code> , <code>predict_features()</code> , <code>evaluate()</code>
main.py	Entry point & benchmarks	<code>load_timit()</code> , <code>benchmark_training()</code> , <code>benchmark_inference()</code> , <code>main()</code>

3. HMM Model Structure

3.1 Model Definition

Each phoneme class is modelled by an independent Gaussian Hidden Markov Model. The model is defined by three sets of parameters:

- Initial state distribution π : $\pi[i]$ = probability of starting in state i at the beginning of a phoneme segment.
- Transition matrix A : $A[i,j]$ = probability of moving from state i to state j at each time step. Each row sums to 1.
- Emission parameters (μ, Σ) : Each state emits feature vectors drawn from a diagonal multivariate Gaussian distribution. $\mu[i]$ is the mean MFCC vector for state i and $\Sigma[i]$ is the per-dimension variance vector.

With 5 states and 13 MFCC features, each HMM has the following parameter counts:

- π : 5 values
- A : $5 \times 5 = 25$ values
- μ : $5 \times 13 = 65$ values
- Σ : $5 \times 13 = 65$ values
- Total per model: 160 parameters
- Total across 60 phoneme classes: 9,600 parameters

3.2 Model Parameters

Parameter	Value Used	Rationale
Number of HMM states (N)	5	Standard for phoneme-level modelling
MFCC coefficients	13	Industry standard for speech features
Emission model	Diagonal Gaussian	Simple, fast, effective baseline
Baum-Welch iterations	25	Sufficient for convergence on TIMIT
Convergence tolerance	1e-4	Early stop if log-likelihood change < tol
Sample rate	16,000 Hz	Native TIMIT recording rate
Min segment length	20 ms	Avoids degenerate very short segments

3.3 Diagonal Gaussian Emission

The log emission probability of observing feature vector x in state i is computed as:

$$\log P(x \mid \text{state } i) = -0.5 * \sum_d [(x_d - \mu_d)^2 / \sigma_d^2 + \log(2\pi\sigma_d^2)]$$

Using diagonal covariance assumes each MFCC dimension is statistically independent given the state. This is a standard simplification that significantly reduces the number of parameters and the risk of overfitting, at the cost of some modelling accuracy.

4. Training: Baum-Welch Algorithm

4.1 Overview

HMM parameters are learned using the Baum-Welch algorithm, which is a specialised instance of the Expectation-Maximisation (EM) algorithm. Starting from random initialisation, it iterates between two steps until convergence:

- E-step: For each training utterance, compute the probability of being in each state at each time frame (gamma) and the probability of each state transition (xi) using the forward-backward algorithm.
- M-step: Update all model parameters (π , A , μ , Σ) using the weighted counts accumulated in the E-step. Each parameter update is a closed-form formula.

4.2 Forward Algorithm

The forward variable $\alpha[t, i]$ represents the joint probability of observing the sequence up to time t and being in state i at time t . It is computed recursively:

```
alpha[0, i] = pi[i] * B[i, o_0]
alpha[t, j] = B[j, o_t] * sum_i ( alpha[t-1, i] * A[i, j] )
```

All computations are performed in log-space using the log-sum-exp trick to prevent floating-point underflow, which would otherwise occur when multiplying hundreds of small probabilities together.

4.3 Backward Algorithm

The backward variable $\beta[t, i]$ represents the probability of observing the remainder of the sequence from time $t+1$ onwards, given state i at time t :

```
beta[T-1, i] = 1 (log-space: 0)
beta[t, i] = sum_j ( A[i, j] * B[j, o_{t+1}] * beta[t+1, j] )
```

4.4 Parameter Update (M-step)

After computing gamma and xi over all training utterances, the parameters are updated as follows:

- $\pi[i] = \text{gamma}[0, i]$ (normalised across states)
- $A[i, j] = \text{sum}_t \text{xi}[t, i, j] / \text{sum}_t \text{gamma}[t, i]$
- $\mu[i] = \text{sum}_t \text{gamma}[t, i] * x_t / \text{sum}_t \text{gamma}[t, i]$
- $\Sigma[i] = \text{sum}_t \text{gamma}[t, i] * x_t^2 / \text{sum}_t \text{gamma}[t, i] - \mu[i]^2$

A variance floor of $1e-6$ is applied to all diagonal covariance values to prevent numerical instability from zero-variance states.

4.5 Convergence

Training continues until the total log-likelihood across all training utterances changes by less than $1e-4$ between iterations, or until 25 iterations are completed. The log-likelihood is guaranteed to be monotonically non-decreasing by the EM convergence theorem, which serves as a built-in correctness check.

5. Inference: Viterbi Decoding

5.1 Algorithm

The Viterbi algorithm finds the single most probable state sequence for a given observation sequence using dynamic programming. It maintains two matrices:

- $\text{delta}[t,i]$: the log-probability of the most likely path ending in state i at time t .
- $\text{psi}[t,i]$: the backpointer recording which state at $t-1$ led to the best path ending in state i at t .

The recursion is:

```
delta[0, i] = log pi[i] + log B[i, o_0]
delta[t, j] = max_i ( delta[t-1, i] + log A[i, j] ) + log B[j, o_t]
```

After processing all frames, the best final state is identified and the full state sequence is recovered by backtracking through the psi matrix from the last frame to the first.

5.2 Classification

For multi-class phoneme recognition, one HMM is trained per phoneme class. To classify a new segment, Viterbi decoding is run against every trained model and the model that produces the highest log-likelihood score is selected as the predicted phoneme:

```
predicted_label = argmax_{class c} score(HMM_c, features)
```

6. Dataset: TIMIT

6.1 Description

The TIMIT Acoustic-Phonetic Corpus is used for all training and evaluation. TIMIT contains 6,300 sentences spoken by 630 speakers from 8 dialect regions of the United States, with time-aligned phoneme transcriptions for every utterance.

6.2 Preprocessing

Audio is processed at the phoneme-segment level rather than the utterance level. For each .WAV file, the corresponding .PHN file provides exact start and end sample numbers for every phoneme. Each phoneme segment is extracted independently and converted to MFCC features:

- Silence segments (h#) are excluded from training and evaluation.
- Segments shorter than 20ms are discarded as too short for reliable feature extraction.
- The FFT window size (n_{fft}) is dynamically reduced for short segments to avoid spectral leakage warnings.
- All MFCC feature vectors are normalised to zero mean and unit variance per dimension.

6.3 Data Statistics

After preprocessing, the dataset contains:

- Training segments: 160,378 phoneme instances across all TRAIN speakers

- Test segments: 58,262 phoneme instances across all TEST speakers
- Unique phoneme classes: 60 (39 standard phonemes + closure symbols + epenthetic silence)
- Average segment length: approximately 60-80ms

7. Results & Baseline Measurements

7.1 Performance Summary

The following results were obtained by running the complete sequential pipeline on the full TIMIT dataset:

Metric	Value	Unit	Notes
Phoneme Classes	60	classes	All TIMIT phonemes incl. closures
Training Segments	160,378	utterances	Segment-level extraction
Test Segments	58,262	utterances	Segment-level extraction
Accuracy	22.4%	%	vs 1.7% random baseline
Total Training Time	3,258.42	seconds	Sequential Baum-Welch
Mean Inference Latency	20.9	ms / segment	Viterbi decode
Inference Throughput	47.5	segments/sec	Sequential single-core
95th Pct Latency	27.3	ms	Upper bound for real-time use

7.2 Accuracy Analysis

The achieved accuracy of 22.4% significantly exceeds the random baseline of 1.7% (1/60 classes), confirming that the HMM is genuinely learning phoneme-specific acoustic patterns. The accuracy is limited by several factors inherent to the baseline model:

- Single Gaussian emission: each HMM state uses one Gaussian, which cannot capture the multi-modal acoustic variability of real phonemes spoken by 630 different speakers.
- No contextual modelling: the system treats each phoneme in isolation. Professional ASR systems use triphone models that condition each phoneme on its left and right neighbours.
- Diagonal covariance: MFCC coefficients are modelled as independent, ignoring their natural correlations.
- No speaker normalisation: the model must explain variation from both acoustic differences between phonemes and between-speaker differences simultaneously.

8. Correctness Verification

The following properties are verified to confirm implementation correctness:

8.1 Monotonic Log-Likelihood

The Baum-Welch algorithm guarantees that the total log-likelihood across all training utterances never decreases between iterations. This was confirmed in all training runs — every iteration showed a positive delta ($\Delta > 0$), consistent with the EM convergence theorem.

8.2 Probability Normalisation

After each M-step, all rows of the transition matrix A sum to 1.0 and the initial state distribution π sums to 1.0, verified programmatically using `numpy.allclose()` with tolerance `1e-4`.

8.3 Viterbi Lower Bound

The Viterbi algorithm finds the single most probable path, while the forward algorithm marginalises over all paths. Therefore the Viterbi log-probability must be $\leq$ the forward log-likelihood for any input. This property holds across all tested segments.

8.4 Deterministic Decoding

Given identical input features, the `predict_features()` function always returns the same predicted label. This confirms there are no random elements in the inference path.

9. Limitations & Future Work

The following limitations are acknowledged in the Milestone 1 baseline and are intentionally not addressed, as the primary goal is parallelisation rather than maximising accuracy:

- Gaussian Mixture Models (GMM): replacing the single Gaussian with a mixture of 8-32 Gaussians per state would substantially improve accuracy and is the standard in production HMM-ASR systems.
- Triphone context: conditioning each phoneme model on its left and right neighbours (triphone models) is the most impactful accuracy improvement but dramatically increases model count and training complexity.
- Delta and delta-delta features: appending velocity and acceleration coefficients to the 13 base MFCCs (giving 39 features total) captures temporal dynamics and is standard practice.
- Speaker normalisation: cepstral mean subtraction per speaker would reduce between-speaker variance and improve accuracy.

These improvements are left for potential future extensions.

10. Conclusion

Milestone 1 delivers a complete, correct, and well-documented sequential HMM speech recognition pipeline trained and evaluated on the full TIMIT dataset. The implementation covers all required components: MFCC feature extraction, Gaussian HMM acoustic modelling, Baum-Welch parameter estimation, and Viterbi decoding. Baseline accuracy and timing measurements have been recorded and will serve as the reference for speedup analysis in Milestones 2 and 3.

Milestone 1 Complete — Sequential Baseline Established — Ready for Parallelisation